

Pruebas de Integración

Truekealo

1. Descripción general

Este documento resume las pruebas de integración del sistema Truekealo, aplicación de trueque. Las pruebas validan flujos end-to-end desde web o móvil hacia la API REST y la persistencia en SQLite.

2. Stack y alcance

- Backend: FastAPI + Python + SQLite
- Frontend: React + Vite
- Mobile: React Native + Expo
- Autenticación/autorización: JWT
- Módulos críticos: usuarios/perfil, artículos, imágenes, mensajería, propuestas y dashboard
- Funciones nativas móvil: cámara, almacenamiento seguro y texto a voz (TTS)
- Accesibilidad: controles y preferencias en web y móvil

3. Objetivo de las pruebas

- Verificar comunicación Frontend/Mobile ↔ Backend mediante API REST.
- Validar flujos completos (registro/login, publicación, mensajería, etc.).
- Confirmar persistencia y consistencia en SQLite (integridad y estados válidos).
- Asegurar manejo correcto de JWT (generación, validación, expiración).
- Comprobar carga y visibilidad de imágenes (multipart + servicio estático).
- Cubrir fallos comunes (401, 404, CORS, errores de red).

4. Configuración previa (mínima)

Requisitos: Python 3.8+, Node.js 16+, Expo CLI, SQLite y requests (pip install requests).

Comandos esenciales:

Backend

- `cd backend`
- `python -m uvicorn app.main:app --reload --host 0.0.0.0`

Frontend (opcional)

- cd frontend
- npm install
- npm run dev

Mobile (opcional)

- cd mobile
- npm install
- npx expo start

Endpoints: Backend <http://localhost:8000> · Web <http://localhost:5173>

5. Catálogo esencial de pruebas

5.1 Backend (Python/FastAPI)

- Autenticación/usuarios: test_auth.py, test_register_api.py, test_profile.py, test_auth_connection.py
- Artículos/imágenes: test_articulos.py, test_articulos_con_imagen.py, test_image_upload.py, test_image_visibility.py, test_mis_articulos.py, test_delete_articulo.py
- Mensajería: test_mensajeria_completa.py, test_mensajeria_completa_v2.py, test_messages_data.py
- Suite general: test_sistema.py, verificar_sistema.py, verificar_sistema_v2.py
- Verificación de datos: verificar_articulos.py, verificar_persistencia.py, verificar_usuario.py
- Servicios: test_email_service.py, test_dashboard_counters.py

```
Responses

Curl
curl -X 'POST' \
  'http://127.0.0.1:8000/api/personas/api/persona/create' \
  -H 'accept: application/json' \
  -H 'content-type: application/json' \
  -d '{
    "nombre": "Jose",
    "apellido": "Riosrio",
    "telefono": "999999999",
    "dni": "1159044489",
    "tipo_dni_id": 1
  }'
```

POST /api/clientes/api/cliente/create Crear Cliente

Parameters Cancel Reset

No parameters

Request body required application/json

Edit Value | Schema

```
{
  "email": "rodrigo.2013.2@gmail.com",
  "password": "123456",
  "contrasena": "1234",
  "estado": true,
  "persona_id": 0
}
```

Execute Clear

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8080/api/clientes/api/cliente/create' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "email": "rodrigo.2013.2@gmail.com",
    "password": "123456",
    "contrasena": "1234",
    "estado": true,
    "persona_id": 0
  }'
```

Request URL

```
http://127.0.0.1:8080/api/clientes/api/cliente/create
```

Server response

POST /api/articulos/api/articulo/create Crear Articulo

Parameters Cancel Reset

No parameters

Request body required application/json

Edit Value | Schema

```
{
  "nombre": "circular",
  "descripcion": "xiomi",
  "estado": true,
  "fecha": "12/11/2025",
  "precio_id": 1,
  "categoria_id": 1
}
```

Execute Clear

Responses

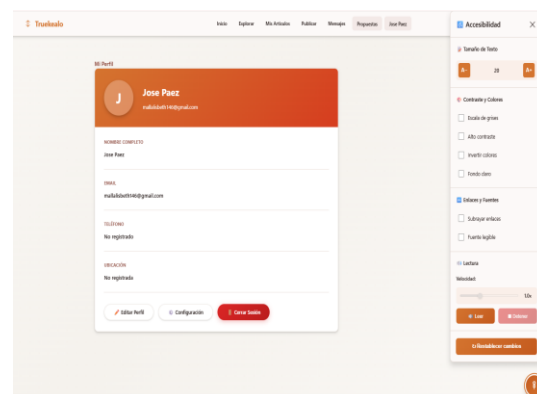
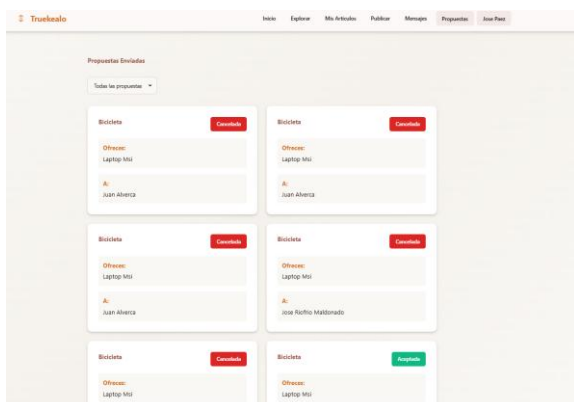
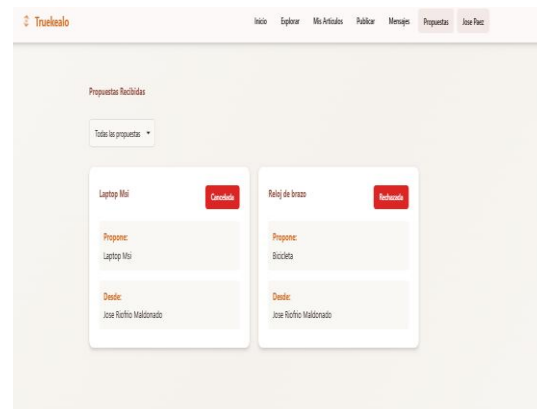
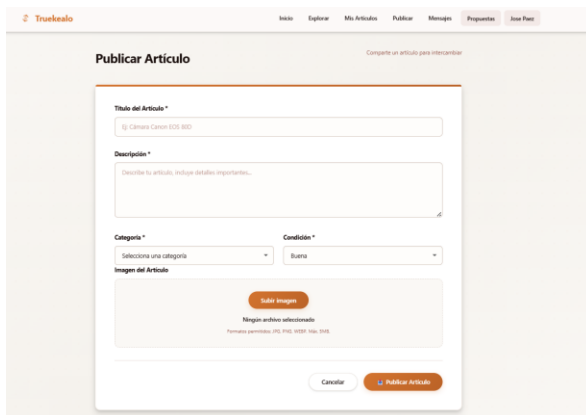
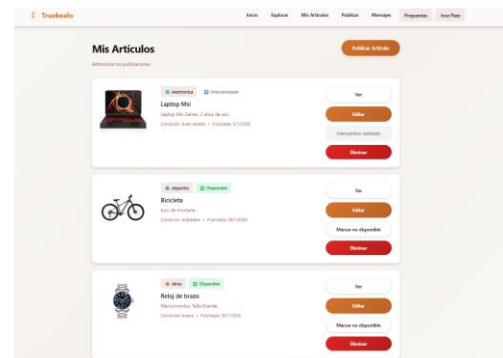
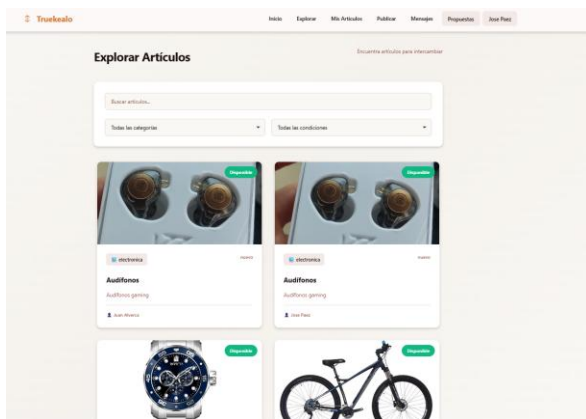
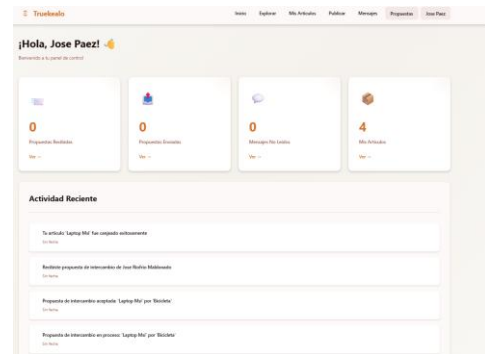
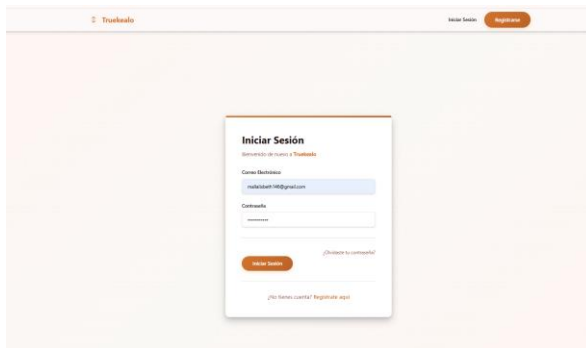
Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8080/api/articulos/api/articulo/create' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "nombre": "circular",
    "descripcion": "xiomi",
    "estado": true,
    "fecha": "12/11/2025",
    "precio_id": 1,
    "categoria_id": 1
  }'
```

5.2 Frontend (React/Vite)

Implementado: pruebas del hook de accesibilidad. Recomendado completar:

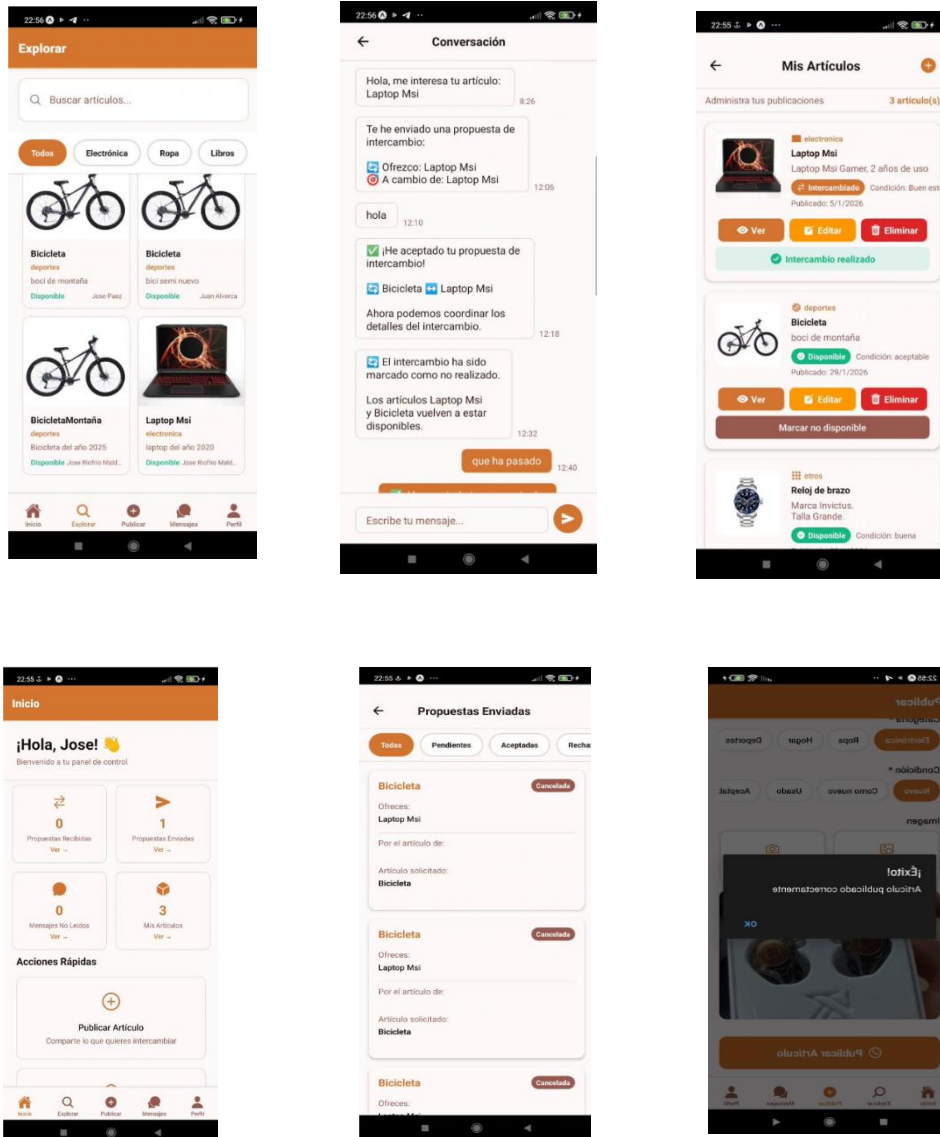
- AuthContext.test.js (sesión, token, expiración, logout)
- ArticuloForm.test.js (validación, preview y subida de imagen)
- Navigation.test.js (rutas activas, responsive, logout)
- api.test.js (headers, 401/404/500, timeout, interceptores)
- IntegrationFlow.test.js (flujo E2E: registrar → login → publicar → mensajería)



5.3 Mobile (React Native/Expo)

Implementado: test-connection.js (salud + login). Recomendado agregar:

- Navigation.test.js (React Navigation: tabs/stacks, deep linking)
- ImagePicker.test.js (cámara/galería + permisos + compresión)
- SecureStorage.test.js (token con SecureStore, fallback web)
- Speech.test.js (TTS: idioma, velocidad, pausa/cancelación)
- MobileIntegration.test.js (flujo E2E móvil)



6. Ejecución total

Existen scripts de ejecución total para Linux/Mac (bash) y Windows (PowerShell) que corren, en orden, pruebas del backend, pruebas del frontend (Jest) y pruebas mobile (Node/Jest).

7. Cobertura resumida

Plataforma	Implementado	Recomendado	Cobertura estimada
Backend	20+ pruebas	-	≈95%
Frontend	1 suite	5 suites	≈20%
Mobile	1 script	5-6 suites	≈15%
TOTAL	22+ pruebas	11 suites	≈60%

8. Problemas típicos y soluciones rápidas

8.1 Backend

- Connection refused: el backend no está corriendo (levantar uvicorn).
- 404 Not Found: prefijo /api/v1/ o baseURL incorrectos.
- 401 Unauthorized: token JWT inválido/expirado o header Authorization faltante.
- Database locked (SQLite): procesos concurrentes; cerrar y limpiar .db-journal si aplica.

8.2 Frontend

- Cannot find module / No tests found: dependencias de testing faltantes o archivos mal nombrados.
- CORS: habilitar CORS en backend y/o proxy en Vite.
- Network request failed: baseURL incorrecta o backend caído.

8.3 Mobile

- Network request failed: usar URL correcta (Android emulator: http://10.0.2.2:8000/api/v1).
- Metro bundler fallando: limpiar caché (expo start -c) y reinstalar node_modules si es necesario.
- Tests con módulos nativos: configurar mocks (expo-image-picker, expo-secure-store).

9. Buenas prácticas mínimas

- Ejecutar pruebas antes de cada commit (ideal con hooks de Git).
- Usar datos de prueba únicos y limpiar usuarios/artículos al finalizar.
- Aislar pruebas para evitar dependencia entre ejecuciones.
- Mockear HTTP en unit tests; reservar llamadas reales para integración/E2E.
- Automatizar en CI/CD y bloquear merges si fallan los tests.